Unit 2

Pointer

A pointer in c is an address, which is a numeric value. Therefore, you can perform arithmetic operations on a pointer just as you can on a numeric value. There are four arithmetic operators that can be used on pointers: ++, --, +, and -

To understand pointer arithmetic, let us consider that **ptr** is an integer pointer which points to the address 1000. Assuming 32-bit integers, let us perform the following arithmetic operation on the pointer −

Incrementing a Pointer

```
#include <stdio.h>

const int MAX = 3;

int main () {

   int  var[] = {10, 100, 200};
   int  i, *ptr;

   /* let us have array address in pointer */
   ptr = var;

   for ( i = 0; i < MAX; i++) {

      printf("Address of var[%d] = %x\n", i, ptr );
      printf("Value of var[%d] = %d\n", i, *ptr );

      /* move to the next location */
      ptr++;
   }

   return 0;
}
```

Decrementing a Pointer

The same considerations apply to decrementing a pointer, which decreases its value by the number of bytes of its data type as shown below −

```
#include <stdio.h>

const int MAX = 3;

int main () {
```

```
   int  var[] = {10, 100, 200};
   int  i, *ptr;

   /* let us have array address in pointer */
   ptr = &var[MAX-1];

   for ( i = MAX; i > 0; i--) {

      printf("Address of var[%d] = %x\n", i-1, ptr );
      printf("Value of var[%d] = %d\n", i-1, *ptr );

      /* move to the previous location */
      ptr--;
   }

   return 0;
}
```

Passing Array to Function in C

In C, there are various general problems which requires passing more than one variable of the same type to a function. For example, consider a function which sorts the 10 elements in ascending order. Such a function requires 10 numbers to be passed as the actual parameters from the main function. Here, instead of declaring 10 different numbers and then passing into the function, we can declare and initialize an array and pass that into the function. This will resolve all the complexity since the function will now work for any number of values.

As we know that the array_name contains the address of the first element. Here, we must notice that we need to pass only the name of the array in the function which is intended to accept an array. The array defined as the formal parameter will automatically refer to the array specified by the array name defined as an actual parameter.

Consider the following syntax to pass an array to the function.

1. functionname(arrayname);//passing array

Methods to declare a function that receives an array as an argument

There are 3 ways to declare the function which is intended to receive an array as an argument.

First way:

1. return_type function(type arrayname[])

Declaring blank subscript notation [] is the widely used technique.

Second way:

1. return_type function(type arrayname[SIZE])

Optionally, we can define size in subscript notation [].

Third way:

1. return_type function(type *arrayname)

passing an array to function example

```
1. #include<stdio.h>
2. int minarray(int arr[],int size){
3. int min=arr[0];
4. int i=0;
5. for(i=1;i<size;i++){
6. if(min>arr[i]){
7. min=arr[i];
8. }
9. }//end of for
10. return min;
11. }//end of function
12.
13. int main(){
14. int i=0,min=0;
15. int numbers[]={4,5,7,3,8,9};//declaration of array
16.
17. min=minarray(numbers,6);//passing array with size
18. printf("minimum number is %d \n",min);
19. return 0;
20. }
```

Output

```
minimum number is 3
```

C function to sort the array

```
1. #include<stdio.h>
2. void Bubble_Sort(int[]);
3. void main ()
4. {
5.     int arr[10] = { 10, 9, 7, 101, 23, 44, 12, 78, 34, 23};
6.     Bubble_Sort(arr);
7. }
```

```
8. void Bubble_Sort(int a[]) //array a[] points to arr.
9. {
10. int i, j,temp;
11.     for(i = 0; i<10; i++)
12.     {
13.         for(j = i+1; j<10; j++)
14.         {
15.             if(a[j] < a[i])
16.             {
17.                 temp = a[i];
18.                 a[i] = a[j];
19.                 a[j] = temp;
20.             }
21.         }
22.     }
23.     printf("Printing Sorted Element List ...\n");
24.     for(i = 0; i<10; i++)
25.     {
26.         printf("%d\n",a[i]);
27.     }
28. }
```

Functions Pointers in C Programming with Examples

Pointers give greatly possibilities to 'C' functions which we are limited to return one value. With pointer parameters, our functions now can process actual data rather than a copy of data.

In order to modify the actual values of variables, the calling statement passes addresses to pointer parameters in a function.

```
1.  #include<stdio.h>
2.  int addition ();
3.  int main ()
4. {
5.      int result;
6.      int (*ptr)();
7.      ptr = &addition;
8.      result = (*ptr)();
9.      printf("The sum is %d",result);
10. }
```

```
11. int  addition()
12. {
13.     int a, b;
14.     printf("Enter two numbers?");
15.     scanf("%d %d",&a,&b);
16.     return a+b;
17. }
```

For example, the next program swaps two values of two:

```
18.     void swap (int *a, int *b);
19.     int main() {
20.       int m = 25;
21.       int n = 100;
22.       printf("m is %d, n is %d\n", m, n);
23.       swap(&m, &n);
24.       printf("m is %d, n is %d\n", m, n);
25.       return 0;}
26.
27.     void swap (int *a, int *b) {
28.       int temp;
29.       temp = *a;
30.       *a = *b;
31.       *b = temp;}
32.     }
```

..